

```
In [ ]: import marimo as mo
```

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção " `box` — grupo genérico de células"),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção " `cube` e `path` "),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção

"Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

box — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor` ou `None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

cube e **path** — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve

ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box`, `cube`, `path`, ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \dots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantires que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);

- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);
- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui`, gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box`, sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube`.
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box`, de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontres.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

```
In [ ]: class Grupo:
    """Conjunto de células com a restrição 'todos diferentes'.
    Guarda (linha, coluna) -> valor fixo, ou None se a célula for livre."""

    def __init__(self, n, cells=None):
        self.n = n
        self.N = n * n
        self.cells = {}
        if cells is not None:
            for (i, j), val in cells.items():
                self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.N and 0 <= j < self.N):
            raise ValueError(f"Célula ({i}, {j}) fora da grelha")
        if val is not None and not (1 <= val <= self.N):
            raise ValueError(f"Valor {val} fora de [1, {self.N}]")
        self.cells[(i, j)] = val

    def matrix(self):
        m = [[0] * self.N for _ in range(self.N)]
        for (i, j), val in self.cells.items():
            if val is not None:
```

```

        m[i][j] = val
    return m

```

```

In [ ]: _g = Grupo(3)
        _g.add(0, 0, 5)
        _g.add(1, 1)

        _erros = []
        for _i, _j, _v in [(9, 0, None), (0, 0, 10), (-1, 2, 3)]:
            try:
                _g.add(_i, _j, _v)
                _erros.append(f"({_i},{_j},{_v}) NÃO foi rejeitado")
            except ValueError:
                pass

        _g2 = Grupo(2)
        _g2.add(3, 3, 4)
        try:
            _g2.add(4, 0)
            _erros.append("(4,0) com n=2 NÃO foi rejeitado")
        except ValueError:
            pass

        mo.callout(
            mo.md(f"cells = `{_g.cells}` \nprimeira linha = `{_g.matrix()[0]}`"
                + (" " if not _erros else f" \n**Falhas:** {_erros}")),
            kind="success" if not _erros else "danger",
        )

```

R1: Grupo (grupo genérico de células)

Correspondência com o enunciado: `box` → `Grupo`. Escolhi este nome porque a classe representa um *grupo de células* sujeito à restrição "todos diferentes", e não a grelha em si.

Ideia central. Num Sudoku a regra é sempre a mesma: um conjunto de células tem de ter valores todos diferentes. O que muda entre uma linha, uma coluna e um bloco é apenas *que células pertencem ao conjunto*. Por isso `Grupo` só sabe guardar "um conjunto de células, algumas já fixas", e não sabe nada sobre linhas, colunas, blocos ou Sudoku. É esta generalidade que permite tratar todos os grupos da mesma forma no modelo CSP (R5) e acrescentar variantes (diagonais, regiões irregulares) sem alterar o modelo.

Estrutura de dados. Usei um dicionário `(linha, coluna) → valor | None`:

- a chave identifica a célula e o valor distingue *célula fixa* (inteiro) de *célula livre* (None), que é exatamente a informação que o enunciado pede;
- só guarda as células que pertencem ao grupo. Um grupo tem tipicamente n^2 células, muito menos do que as n^4 da grelha, por isso uma matriz $n^2 \times n^2$ por

grupo desperdiçaria memória (no caso de $n = 6$, seriam 1296 posições por cada um dos $3n^2 = 108$ grupos);

- acesso e inserção em tempo médio constante, e uma célula repetida limita-se a sobrescrever a entrada anterior em vez de ficar duplicada.

Parametrização. A classe guarda n e calcula $N = n^2$ uma única vez. Todos os limites (coordenadas em $[0, N - 1]$, valores em $[1, N]$) dependem de N , e nunca de um 9 escrito no código. Isto garante que funciona para qualquer n (testado com $n = 2$ e $n = 3$).

Validação. O método `add` rejeita, com `ValueError`, coordenadas fora da grelha e valores fora de $[1, n^2]$, e só depois guarda a célula. Assim, um grupo nunca fica num estado inválido. O construtor, quando recebe um conjunto inicial de células, chama `add` para cada uma, de modo que a validação existe num único sítio.

Representação matricial. `matrix()` devolve uma matriz $n^2 \times n^2$ com o valor fixo nas células fixas e `0` em todas as outras (células livres e células fora do grupo). Aqui perde-se a distinção entre "livre" e "não pertence", mas isso é intencional: a matriz serve para mostrar valores, e a distinção continua disponível no dicionário `cells`, que é o que o modelo CSP usa.

```
In [ ]: class Cube(Grupo):
        """Bloco n x n de índices (bi, bj); canto superior esquerdo em (bi*n,
        bj*n)."""

        def __init__(self, n, bi, bj):
            super().__init__(n)
            if not (0 <= bi < n and 0 <= bj < n):
                raise ValueError(f"Bloco ({bi}, {bj}) fora de [0, {n})")
            for di in range(n):
                for dj in range(n):
                    self.add(bi * n + di, bj * n + dj)
```

```
In [ ]: _b = Cube(3, 1, 2)
assert len(_b.cells) == 9
assert min(i for i, _ in _b.cells) == 3 and max(i for i, _ in _b.cells) == 5
assert min(j for _, j in _b.cells) == 6 and max(j for _, j in _b.cells) == 8

_b2 = Cube(2, 1, 1)
assert sorted(_b2.cells) == [(2, 2), (2, 3), (3, 2), (3, 3)]

try:
    Cube(3, 3, 0)
    _rejeitou = False
except ValueError:
    _rejeitou = True
assert _rejeitou
```

```
mo.callout(mo.md("`Cube` OK para $n=3$ e $n=2$, e rejeita blocos inválidos."), kind="success")
```

R2: Cube (bloco $n \times n$)

Correspondência com o enunciado: `cube` $\rightarrow$ `Cube`.

Ideia central. Um bloco $n \times n$ é só um grupo de células, por isso `Cube` herda de `Grupo`. A herança é adequada porque um `Cube` é um `Grupo` (tudo o que se pode fazer com um grupo faz-se com um bloco) e porque o modelo CSP vai tratá-lo exatamente como qualquer outro grupo, sem precisar de saber que é um bloco.

Construção. O bloco de índices (b_i, b_j) , com $0 \leq b_i, b_j < n$, tem o canto superior esquerdo em $(b_i \cdot n, b_j \cdot n)$. O construtor chama `super().__init__(n)` para criar o grupo vazio e depois percorre os deslocamentos $d_i, d_j \in [0, n)$, adicionando a célula $(b_i \cdot n + d_i, b_j \cdot n + d_j)$. São n^2 células, que é precisamente o tamanho de um bloco e o número de valores distintos que o bloco tem de conter.

Reutilização e validação. `Cube` não repete nenhuma lógica de `Grupo`: usa `add`, que já valida as coordenadas. Acrescentei apenas a validação de (b_i, b_j) , porque sem ela um índice de bloco inválido seria rejeitado pelo `add` com uma mensagem sobre uma célula fora da grelha, que confunde quem estiver a pensar em *blocos*.

Generalidade. Nada no código assume $n = 3$: os índices e o tamanho do bloco dependem só de n . Para $n = 2$ obtêm-se 4 blocos 2×2 numa grelha 4×4 , e para $n = 3$ obtêm-se 9 blocos 3×3 numa grelha 9×9 (ambos verificados nos testes).

```
In [ ]: class Path(Grupo):
    """Troço reto (horizontal ou vertical) de inicio a fim, inclusive, em
    qualquer sentido."""

    def __init__(self, n, inicio, fim):
        super().__init__(n)
        (i0, j0), (i1, j1) = inicio, fim
        if i0 != i1 and j0 != j1:
            raise ValueError("inicio e fim têm de estar na mesma linha ou
            coluna")

        def _sinal(x):
            return (x > 0) - (x < 0)

        di, dj = _sinal(i1 - i0), _sinal(j1 - j0)
        tamanho = max(abs(i1 - i0), abs(j1 - j0)) + 1
        for k in range(tamanho):
            self.add(i0 + k * di, j0 + k * dj)
```

```
In [ ]: assert list(Path(3, (0, 5), (0, 2)).cells) == [(0, 5), (0, 4), (0, 3), (0, 2)]
assert list(Path(3, (2, 1), (5, 1)).cells) == [(2, 1), (3, 1), (4, 1), (5,
```

```

1)]
assert list(Path(3, (4, 4), (4, 4)).cells) == [(4, 4)]
assert len(Path(2, (1, 0), (1, 3)).cells) == 4

for _ini, _fim in [((0, 0), (1, 1)), ((0, 0), (0, 9))]:
    try:
        Path(3, _ini, _fim)
        raise AssertionError(f"{_ini}->{_fim} devia ser rejeitado")
    except ValueError:
        pass

mo.callout(mo.md("`Path` OK nos dois sentidos, horizontal e vertical, e rejeita casos inválidos."), kind="success")

```

R3: Path (troço reto de células)

Correspondência com o enunciado: `path` → `Path`.

Ideia central. Uma linha, uma coluna, ou qualquer troço reto de células, é apenas um grupo de células. Por isso `Path`, tal como `Cube`, herda de `Grupo` e limita-se a preencher as células: o modelo CSP trata-o como qualquer outro grupo, sem precisar de saber que é um troço reto.

Construção. Dados `inicio` = (i_0, j_0) e `fim` = (i_1, j_1) , o troço inclui ambos os extremos. Em vez de tratar separadamente os quatro sentidos possíveis, calculo o passo em cada eixo como o **sinal** da diferença:

$$d_i = \text{sgn}(i_1 - i_0), \quad d_j = \text{sgn}(j_1 - j_0)$$

O sinal vale 1, 0 ou -1 , e o troço tem $\max(|i_1 - i_0|, |j_1 - j_0|) + 1$ células, a k -ésima das quais é $(i_0 + k d_i, j_0 + k d_j)$. Se `fim` vem *depois* de `inicio`, os passos são positivos; se vem *antes*, são negativos, e o troço é percorrido em sentido inverso, como o enunciado exige. Se os dois pontos estão na mesma linha, $d_i = 0$ e só a coluna varia (e vice-versa), pelo que não é preciso nenhum `if` por direção.

Casos limite.

- Se `inicio` = `fim`, o tamanho é 1 e o grupo tem uma única célula.
- Se os pontos não estão na mesma linha nem na mesma coluna (uma diagonal), o construtor levanta `ValueError`. Sem esta verificação, o código construiria uma diagonal sem avisar, o que seria um erro silencioso.
- Coordenadas fora da grelha são rejeitadas por `add`, herdado de `Grupo`.

Utilização no Sudoku. Uma linha completa i é `Path(n, (i, 0), (i, N-1))` e uma coluna completa j é `Path(n, (0, j), (N-1, j))`, com $N = n^2$. Há n^2 linhas e n^2 colunas, e em ambos os casos o grupo tem n^2 células.


```
In [ ]: import random

def pistas(n, k=None, rng=random):
    """Grupo com k células escolhidas ao acaso, cada uma fixa a um valor
    aleatório.
    Os valores são todos distintos (por isso k <= N)."""
    N = n * n
    if k is None:
        k = n
    if not (0 <= k <= N):
        raise ValueError(f"k={k} fora de [0, {N}]")
    posicoes = [(i, j) for i in range(N) for j in range(N)]
    escolhidas = rng.sample(posicoes, k)
    valores = rng.sample(range(1, N + 1), k)
    g = Grupo(n)
    for (i, j), v in zip(escolhidas, valores):
        g.add(i, j, v)
    return g
```

```
In [ ]: for _n in (2, 3):
    _g = pistas(_n)
    assert len(_g.cells) == _n
    _vals = list(_g.cells.values())
    assert all(v is not None for v in _vals)
    assert len(set(_vals)) == len(_vals)
    assert all(1 <= v <= _n * _n for v in _vals)
assert len(pistas(3, k=9).cells) == 9
try:
    pistas(3, k=10)
    raise AssertionError("k=10 devia ser rejeitado")
except ValueError:
    pass
mo.callout(mo.md("`pistas` OK para $n=2$ e $n=3$."), kind="success")
```

R4: pistas aleatórias

Resultado. `pistas(n, k)` devolve um `Grupo`, sem classe nova.

Escolha das células. Uso `random.sample` sobre a lista de todas as posições, que escolhe `k` posições *sem repetição*. Com escolhas repetidas, a mesma célula poderia sair duas vezes e `add` limitar-se-ia a sobrescrevê-la.

Escolha dos valores. O modelo impõe "todos diferentes" a *cada* grupo, e o grupo de pistas é um grupo como os outros. Se dois valores sorteados fossem iguais, o puzzle ficaria **sempre** sem solução. Por isso sorteio também os valores sem repetição, o que exige $k \leq n^2$.

Valor por omissão. $k = n$, da ordem de n como o enunciado sugere.

Puzzle sem solução. [Escreve aqui a tua decisão: tentar novas pistas ou reportar o insucesso, e porquê.]

```
In [ ]: from ortools.linear_solver import pywraplp
```

```
class Modelo:
    def __init__(self, n):
        self.n = n
        self.N = n * n
        self.solver = pywraplp.Solver.CreateSolver("SCIP")
        self.x = {}
        for i in range(self.N):
            self.x[i] = {}
            for j in range(self.N):
                self.x[i][j] = {}
                for v in range(1, self.N + 1):
                    self.x[i][j][v] = self.solver.IntVar(0, 1,
f"x_{i}_{j}_{v}")
                    # restrição 1: cada célula tem exatamente um valor
                    # completar

    def X(self, i, j, v):
        return self.x[i][j][v]

    def add_grupos(self, *grupos):
        for g in grupos:
            # restrição 2: completar
            # restrição 3: completar
            ...

    def resolver(self):
        stat = self.solver.Solve()
        # se houver solução: devolver a matriz N x N com o valor de cada
        célula
        # se INFEASIBLE: devolver None
        ...
```